

PCD - Programação Concorrente e Distribuída

Coordenação de processos ligeiros - Variáveis Condicionais e Semáforos

Luís Mota

October 21, 2024

Estrutura

- 1 Cadeados
- 2 Variáveis condicionais
- 3 Fila bloqueante
- 4 Semáforo

Interface Lock

Lock apresenta vantagens relativamente aos cadeados intrínsecos:

- operações de trancar e destrancar;
- trancar cadeado com possibilidade de a operação ser interrompida;
- simples tentativa de trancar: se não estiver disponível, devolve o valor `false`;
- associa *variáveis condicionais* ao cadeado.

Atenção!

Cadeados devem ser usados como alternativa à sincronização e não em simultâneo: tal pode facilmente originar bloqueios!

Interface Lock: métodos

`void lock/unlock` operações com o comportamento habitual;

`void lockInterruptibly()` tranca o cadeado a menos que seja interrompido;

`boolean tryLock()` tranca o cadeado apenas se estiver disponível, senão devolve `false`;

`boolean tryLock(long time, TimeUnit unit)` tranca o cadeado apenas se estiver disponível dentro do prazo indicado e não tiver sido interrompido, senão devolve `false`.

Manipulação de cadeados: na prática

```
private Lock l = ...; // atributo!  
// (...)  
l.lock();  
try {  
    // aceder e manipular o recurso  
    // protegido pelo cadeado  
    // (...)  
} finally {  
    l.unlock();  
}
```

Recapitulando: Métodos sincronizados

```
class Pilha {  
    private int pos = -1;  
    private int pilha[]= new int[100];  
    public synchronized void push(int i) {  
        pos++;  
        pilha[pos]=i;  
    }  
    public synchronized int pop() {  
        if (pos>=0){  
            return(pilha[pos--]);  
        } else  
            throw new IllegalStateException() ;  
    }  
}
```

Pilha usando cadeado

```
public class Pilha {  
    private int pos = -1;  
    private int pilha[] = new int[1000000];  
    private Lock l = new ReentrantLock();  
    public void push(int i) {  
        l.lock();  
        try {  
            pos++;  
            pilha[pos] = i;  
        }  
        finally {  
            l.unlock();  
        }  
    }  
    //(...)  
}
```

Pilha usando cadeado

```
public class Pilha {  
    private int pos = -1;  
    private int pilha[] = new int[100000];  
    private Lock l = new ReentrantLock();  
    // (...)  
    public int pop() {  
        l.lock();  
        try {  
            if (pos >= 0) {  
                return pilha[pos--];  
            } else  
                throw new IllegalStateException();  
        }  
        finally {  
            l.unlock();  
        }  
    }  
}
```


Estrutura

- 1 Cadeados
- 2 Variáveis condicionais
- 3 Fila bloqueante
- 4 Semáforo

Variáveis condicionais

- Nome é algo enganador: não testam nenhuma condição... Mas, no Java, chamam-se `Condition`
- Talvez fosse mais intuitivo serem designadas 'Situação' ou 'Evento';
- Estão associadas a um cadeado (`Lock`) e só podem:
 - ser criadas a partir do cadeado;
 - ser manipuladas quando se detém o cadeado.
- Variáveis condicionais estão tipicamente ligadas às situações em que uma operação deixa de estar bloqueada e pode prosseguir normalmente!

Guião de uso de variáveis condicionais

- ❶ Adquirir (lock) o cadeado a que a variável condicional está associada;
- ❷ verificar num `while` se não estão satisfeitas as condições para prosseguir a operação;
 - nesse caso, esperar (`await`) que a variável condicional seja *assinalada*.
- ❸ senão, avançar com a execução e, no fim, libertar o cadeado;
- ❹ Outra *thread* que *satisfaça* a variável condicional deverá *assinalar* (`signal`) esse facto.
- ❺ Quando uma *thread* é acordada, deve voltar a verificar se as condições de avançar estão reunidas. (Porquê?)

Por precaução: várias *threads* podem ter sido acordadas e alguma outra já ter ocupado os recursos disponíveis, “*Spurious wakeup*”, etc.

Exemplo de variáveis condicionais

Pretende-se modelar um visor de encomendas de take-away, que será neste caso o *Recurso Partilhado*. Existem dois tipos de processos ligeiros: o cozinheiro e o estafeta, que podem ser em número variável.

- As encomendas podem chegar do estafeta a qualquer momento, mas apenas podem ser exibidas quando o visor estiver livre. Se um estafeta chegar e o visor estiver ocupado (pois nenhum cozinheiro tomou ainda conta da encomenda já aí exibida), a aceitação da encomenda deve ficar em espera.
- Quando um cozinheiro retira uma encomenda do visor, este passa a estar livre.
- Se o cozinheiro tentar retirar uma encomenda e nenhuma estiver disponível, deve ficar em espera.

Condições de bloqueio geram variáveis condicionais

As variáveis condicionais são a **negação** das condições de bloqueio!

Operação	Bloqueio	Variável condicional
publishOrder	há encomenda no visor	visor vazio screenEmpty
takeOrder	visor vazio	há encomenda no visor orderOnScreen

Código do visor

```
public class OrderScreen {
    private Lock lock= new ReentrantLock();
    private Condition orderOnScreen = lock.newCondition();
    private Condition screenEmpty= lock.newCondition();
    private String order=null;
    public void publishOrder(String newOrder)
        throws InterruptedException{
        lock.lock();
        try{
            while(order!=null)
                screenEmpty.await();
            order=newOrder;
            orderOnScreen.signalAll();
        }finally{
            lock.unlock();
        }
    }
}
```

Código do visor

```
public String takeOrder() throws InterruptedException{
    String res=null;
    lock.lock();
    try{
        while(order==null)
            orderOnScreen.await();
        res=order;
        order=null;
        screenEmpty.signalAll();
    }finally{
        lock.unlock();
    }
    return res;
}
```

Vantagens das variáveis condicionais

Que vantagens há em usar variáveis condicionais em relação ao mecanismo de coordenação com `wait/notifyAll`?

Assim pode evitar-se estar repetidamente a acordar **todas** as *threads* em espera, dando-se um sinal **apenas** às que estão interessadas num determinado evento/condição.

Estrutura

- 1 Cadeados
- 2 Variáveis condicionais
- 3 Fila bloqueante
- 4 Semáforo

Fila bloqueante

- Nas bibliotecas Java, existe a interface `BlockingQueue`, com várias implementações.
- Trata-se de uma fila que coloca em espera as *threads* se:
 - estiver vazia quando se pretende retirar (método `take`), ou
 - se não existir mais capacidade disponível ao colocar um novo elemento (método `put`).

Implementação da BlockingQueue

Vamos otimizar a implementação, usando variáveis condicionais.
Esqueleto da classe:

```
class BlockingQueue<E>{  
    private Queue<E> data = new ...  
    public void put(E value) throws InterruptedException  
    public E take() throws InterruptedException  
}
```

Quais as possíveis condições de bloqueio?

Condições de bloqueio geram variáveis condicionais

As variáveis condicionais são a **negação** das condições de bloqueio!

Operação	Bloqueio	Variável condicional
put	está cheia	notFull
take	está vazia	notEmpty

Atributos de BlockingQueue

```
private Queue<E> queue=new LinkedList<E>();  
private Lock lock= new ReentrantLock();  
private Condition notEmpty=lock.newCondition();  
private Condition notFull=lock.newCondition();  
private final int capacity;
```

Método put

```
public void put(E elem) throws InterruptedException{  
    lock.lock();  
    try{  
        while(queue.size()==capacity) // is full!  
            notFull.await();  
        queue.offer(elem);  
        notEmpty.signalAll();  
    }finally{lock.unlock();}  
}
```

Método take

```
public E take() throws InterruptedException{  
    lock.lock();  
    try{  
        while(queue.isEmpty())  
            notEmpty.await();  
        E elem=queue.poll();  
        notFull.signalAll();  
        return elem;  
    }finally{lock.unlock();}  
}
```

Estrutura

- 1 Cadeados
- 2 Variáveis condicionais
- 3 Fila bloqueante
- 4 Semáforo

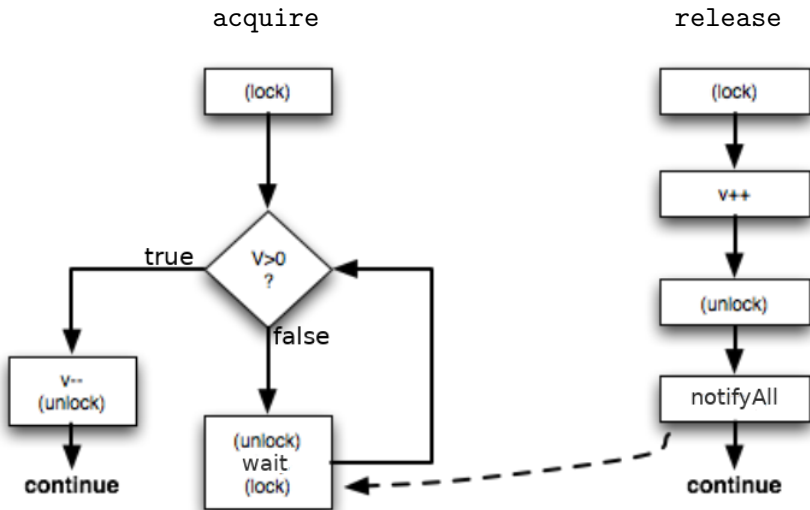
Semáforo

- Semáforo é um conceito de coordenação, disponibilizando um número de *licenças*, que pode ser incrementado arbitrariamente, mas não pode assumir valores abaixo de zero.
- É útil para fazer *threads* esperar por eventos que acontecem em sucessão, ou para limitar acesso a recursos.

Operações sobre semáforos

- construtor** : recebe o número de *licenças* existentes inicialmente (mas não existe máximo). Opcionalmente, na definição existente na biblioteca Java, pode ser parametrizado como `fair`: tentará atribuir as licenças por ordem de chegada do pedido.
- acquire** : adquire uma licença do semáforo. Se estas não estiverem disponíveis (i.e., a zero), bloqueia até haver alguma licença disponível.
- release** : liberta uma licença para o semáforo e notifica possíveis *threads* bloqueadas na operação `acquire`

Operações sobre semáforos



Implementação de Semáforo

```
public class Semaphore {  
    int permits = 0;  
    public Semaphore(int i) {  
        permits = i;  
    }  
    public Semaphore() {  
        permits = 0;  
    }  
    //...  
}
```

Implementação de Semáforo

```
public synchronized void acquire()  
    throws InterruptedException{  
    while (permits == 0)  
        wait();  
    permits--;  
}  
  
public synchronized void release() {  
    permits++;  
    notifyAll();  
}
```

Aquisição ininterruptível

A classe do Java Semaphore disponibiliza também o método `acquireUninterruptibly`:

- em caso de não haver licenças disponíveis, o método também bloqueia, mas sem poder ser interrompido;
- porém, se de facto existir uma interrupção durante o bloqueio, à saída do método a *flag* de interrupção estará ativa.

Desafio: Semáforo justo

Como alterar a implementação do semáforo para que a atribuição de licenças seja (aproximadamente) *justa*, i.e., por ordem de chegada do pedido?

Esta questão vai surgir no enunciado do laboratório desta semana!

- 1 Cadeados
- 2 Variáveis condicionais
- 3 Fila bloqueante
- 4 Semáforo